FINAL PROJECT REPORT

EasyRTG

An Intelligent No-Code Platform for Training Robots

Shivam Chavan
Student ID: 3053682

Supervisor: Shraddha Naik

MSc in Artificial Intelligence and Computer Science
School of Computer Science
University of Birmingham Dubai
August 2026

Declaration and Honour Code

I certify that the work presented in this report is my own. The design decisions, the implementation, the experimental work, the interpretation of the results and the conclusions drawn from them are mine, and together they describe the EasyRTG system held in the GitLab repository submitted alongside this document. Where I have relied on the ideas, the software or the published findings of other authors, I have acknowledged that reliance through numbered citations in the text and corresponding entries in the bibliography.

I understand that the final submission must comply with the University's academic-integrity requirements, including the prescribed honour-code wording and the disclosure of any supporting tools used during the work. My disclosure of generative-AI use is given on the following page.

Name: Shivam Chavan

Student ID: 3053682

Use of Generative AI

Generative AI was used as a supporting tool during this project, but it was not the author of either the system or this report. Its contribution was confined to the four areas set out below.

- Debugging. When a simulator configuration failed to load, or when a policy converged on an unintended behaviour, I used an assistant to help interpret stack traces and to narrow down the list of candidate causes. Every fix arrived at in this way was verified against the running system before I retained it.
- Exploring implementation options. I asked for comparisons between approaches that I was already considering, such as whether Isaac Sim should be embedded in the backend process or launched as a subprocess. The decisions themselves, and the reasoning recorded in Chapter 5, are my own.
- Boilerplate and documentation. I used an assistant to draft repetitive code and docstrings, which allowed me to concentrate my own effort on the morphology, expression and audit layers that carry the technical contribution of the project.
- Language and formatting. I used it to tighten wording and to assist in preparing this document for submission. The structure of the argument, and the claims made within it, remain mine.

Every suggestion I accepted was checked against the source code, the repository history, the recorded telemetry or the cited literature, depending on which was relevant. Where a generated suggestion disagreed with the observed behaviour of the system, I retained the behaviour of the system. Responsibility for the technical content of this report, and for the submitted software, is mine alone.

Acknowledgements

I would like to express my gratitude to my supervisor, Shraddha Naik for her guidance over this project. This is mainly thanks to her feedback, as when I came up with a new meaningful features, she asked what meaning that feature would actually create. That question ended up shaping the whole second half of the work.

I also owe thanks to the School of Computer Science for the environment and resources that have enabled the work to be carried out, specifically the GPU workstation which has allowed the parallel training experiments detailed in Chapter 6 to be attempted.

Last but not least, I would like to thank the people who encouraged me in the parts of this project that never showed up in a demonstration, such as the simulator configurations, which refused to load, the policies, which learned to crawl instead of walk, and the overnight running of simulators, whose difficulties only became apparent near the end. While annoying at the time, many of the design decisions documented in this report stem from these episodes.

Abstract

Typically, training a robot with reinforcement learning involves putting together six pieces of software: a physics simulator, an environment implementation, a reward function, a learning library, some way to monitor experiments and some way to evaluate the resulting policy. Every one of these components takes the other five for granted, and that is why robot learning is not accessible to a newcomer: the barrier is not the algorithms, but the circle of skills each piece assumes you already have. EasyRTG was created to collapse that assembly into a single guided workflow. The user simply uploads a robot in Universal Robot Description Format (URDF), specifies the desired behaviour in natural language, checks out a proposed configuration generated by an assistant, validates the proposed reward without using up GPU time, initiates parallel training, and replays the resulting policy within the app, without needing to write any simulator or reinforcement-learning code.

The system put into place consists of four components: a Flutter front-end desktop interface, a FastAPI orchestration back-end, a PyBullet robot inspection and reward testing framework and finally, a parallel training and playback framework based on NVIDIA Isaac Sim. The main technical decision it has made is to get rid of the hard coded robot definitions. Instead of joint-name rules and a fixed menu of reward terms, EasyRTG derives the control-relevant morphology from the URDF itself and represents every reward term as an audited mathematical expression over a common state vocabulary. This is what enabled one application pipeline to train both quadruped and humanoid robots without having to make special modifications to the application to account for each different body.

The platform was assessed across 40 timestamped Isaac run directories collected between 13 July and 18 August 2026, of which 34 contained usable reward telemetry and six failed before producing any. The experiments include the Unitree A1, ANYmal and Unitree Go1 as well as the NASA Valkyrie, under PPO, SAC and TD3. Twelve of the 34 analysable runs held at least 95% of the episode limit of 1,000 steps, and the highest throughput recorded was 32,056.8 environment steps per second on a single RTX 5000. On Go1, the three learners reached 96.3%, 100% and 99.1% of the episode cap under PPO, SAC and TD3 respectively. On Valkyrie, with the same reward expression and the same step budget, SAC reached 82.1% of the cap while PPO and TD3 reached only about 20%, so the difference in performance is due to the choice of learner alone.

These findings cut both ways, and this report says so. GPU parallelism enabled the experimental programme to be feasible in the first place, but it also made it easy to obtain a high reward which was not linked to the desired behaviour. This happened on the galloping task, where TD3 scored well with a fast trot that never left the ground. What EasyRTG proves is a working integration of language-assisted configuration, general robot inspection and accelerated learning. It does not yet show a controlled decrease in cognitive load, which would need a user study, nor does it show transfer to physical hardware. The next stage of the work is defined by these two omissions.

Table of Contents

1 Introduction 1

1.1 Project motivation 1

1.2 Problem statement 1

1.3 Aim and objectives 2

1.4 Research questions 2

1.5 Scope and contributions 3

1.6 Organisation of this report 3

2 Background and Literature Survey 3

2.1 Reinforcement learning for robot control 3

2.2 The three learners: PPO, SAC and TD3 4

2.3 From single environments to massively parallel simulation 4

2.4 Isaac Gym and the GPU-native turn 5

2.5 Orbit and Isaac Lab 5

2.6 Learning to walk in minutes 5

2.7 Reward design as the real bottleneck 5

2.8 Eureka: language models writing rewards 6

2.9 DrEureka: pushing language-assisted design toward hardware 6

2.10 General-purpose simulators: Gazebo and Unity ML-Agents 6

2.11 Unity RL Playground: the closest accessibility claim 7

2.12 Where EasyRTG sits 7

3 Research Methodology and Requirements 8

3.1 Design-science approach 8

3.2 Evidence sources 8

3.3 Functional and non-functional requirements 9

3.4 Experimental protocol and metrics 9

3.5 Software verification 10

4 System Design and Architecture 10
4.1 Architectural overview 10
4.2 URDF inspection and morphology 11
4.3 Expression rewards and the shared state vocabulary 11
4.4 Assistant and human oversight 12
4.5 Training, monitoring and evaluation 13
5 Implementation and Iterative Development 14
5.1 From proposal to working system 14
5.2 Milestone history 14
5.3 Key engineering decisions 16
5.4 Security and reliability improvements 16
6 Experiments, Results and Findings 16
6.1 Run coverage and failure history 16
6.2 Fleet-level results 17
6.3 What the reward actually paid for 18
6.4 Throughput and the GPU lane 19
6.5 How long learning took 19
6.6 Go1: three learners, one solved robot 20
6.7 Go1 walking and galloping 21
6.8 Valkyrie: a controlled algorithm comparison 22
6.9 Functional verification 23
7 Discussion 24
7.1 Answering the research questions 24
7.2 What the project learned about abstraction 25
7.3 Limitations and threats to validity 25
7.4 Practical and academic significance 26
8 Conclusion and Future Work 26

8.1 Conclusion 26

8.2 Future work 27

Bibliography 27

Appendix A GitLab Repository and Running the Project 29

A.1 Repository 29

A.2 Minimum setup 29

A.3 Verification commands 30

Appendix B Experiment Coverage 30

Appendix C Supplementary Figures 30

Appendix D Full Run Index 33

Appendix E Compute Environment and Reproduction Notes 34

List of Figures

Figure 1. The EasyRTG workflow, from robot import to policy evaluation. 1
Figure 2. EasyRTG architecture, and the separation between the inspection and GPU training lanes. 11
Figure 3. The EasyRTG desktop assistant, showing a generated training configuration and the changes offered for review. 13
Figure 4. Major implementation and experiment milestones, reconstructed from Git dates and the run archive. 14
Figure 5. Episode survival for the best observed run in each robot and algorithm pairing. 17
Figure 6. Term-by-term decomposition of the reward at the peak-reward step of the best Go1 run. 18
Figure 7. Peak simulation throughput against the number of parallel environments, coloured by robot. 19
Figure 8. Reward for the best Go1 run under each learner, at 4,096 environments with the same reward expression. 20
Figure 9. The Unitree Go1 during recorded policy evaluation. 21
Figure 10. Reward for Valkyrie under SAC and TD3, same reward expression and same 40M-step budget. 22
Figure 11. Episode length reveals that SAC's advantage came from staying upright, not from scoring faster. 22
Figure 12. The NASA Valkyrie during recorded SAC policy evaluation. 23
Figure 13. Best mean episode length for every analysable Isaac run, in chronological order. 31
Figure 14. Reward against environment steps for the best run of each robot family. 31
Figure 15. Environment steps before each run's smoothed mean episode length first holds above 900. 32
Figure 16. Episode length for the best Go1 run under each learner. 32

List of Tables

Table 1. Research questions used to organise the implementation and evaluation. 2
Table 2. Algorithm rationale and its observed role in EasyRTG. 4
Table 3. Positioning EasyRTG against representative robot-simulation and learning platforms. 7
Table 4. Traceable project requirements. 9
Table 5. Engineering decisions that emerged through iteration. 16
Table 6. Best observed run for each robot and algorithm pairing. 17
Table 7. Coverage of the 34 analysable Isaac runs by robot and algorithm. 30
Table 8. All 34 analysable Isaac runs, 13 July to 18 August 2026. 33
Table 9. Compute and software environment used for all experiments. 34

Abbreviations

Term	**Meaning**
API	Application Programming Interface
AST	Abstract Syntax Tree
DRL	Deep Reinforcement Learning
GPU	Graphics Processing Unit
LLM	Large Language Model
PPO	Proximal Policy Optimisation
REST	Representational State Transfer
RL	Reinforcement Learning
SAC	Soft Actor-Critic
TD3	Twin Delayed Deep Deterministic Policy Gradient
UI	User Interface
URDF	Universal Robot Description Format
USD	Universal Scene Description

1 Introduction

1.1 Project motivation

The concept behind EasyRTG stemmed from an earlier robotics project of mine that was a success, but a less-than-informative one. The demonstration was convincing enough and the robot did learn the behaviour I wanted it to. What the demonstration did not show was how much manual work had gone into connecting the simulator, the environment code, the reward function, the training scripts and the visualisation. After a few months of that, the difficult part of the problem was no longer the robot at all; it was the knowledge needed to make the surrounding tools agree with each other.

It was that experience that pointed me at the real barrier. Reinforcement learning is generally presented as a small cycle in which an agent observes a state, takes an action and receives a reward [1]. A new user attempting to use it, however, must also handle the simulator setup, the robot file format, coordinate systems, action scaling, termination conditions, hyperparameters, logging and evaluation. Mature libraries decrease the amount of algorithm code that has to be written, but in general the user is still expected to be able to build a valid environment already [2]. So I approached it from another direction and forced myself to ask a different question: what happens if the process of creating the environment becomes the product itself, instead of an undocumented prerequisite?

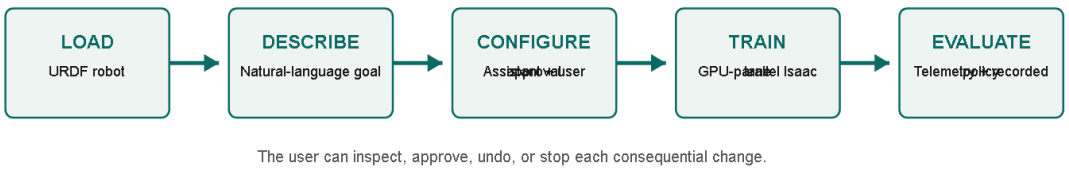


Figure 1. The EasyRTG workflow, from robot import to policy evaluation.

1.2 Problem statement

What this project addresses is not a lack of reinforcement-learning algorithms. Current frameworks are powerful, but that power is only reachable through source code, configuration files and conventions peculiar to each simulator. Loading a URDF on its own tells you nothing: not which joints should be controlled, not which links may touch the ground, not how high the body should spawn, and certainly not which reward distinguishes walking from crawling. The

problem statement of this project is therefore the fragmented expertise needed to turn a robot description and a statement of behavioural intent into a training run which is valid, inspectable and repeatable.

1.3 Aim and objectives

My desired end goal was to build a desktop platform that takes the user from a URDF robot and an objective stated in natural language to a trained and evaluated policy, without having to write simulator or reinforcement-learning code. To support that I set five objectives:

- Single workflow for robot inspection, configuration, monitoring, training and evaluation directly on a desktop.
- Retrieve the morphology required for control directly from the URDF, without relying on robot-specific rules about the naming of joints.
- Enable an LLM assistant to author reward expressions and configuration changes, while keeping every consequential action reviewable and approved by the user.
- GPU-parallel simulation, so that repeated robot-learning experiments are practical on a single workstation.
- Evaluate the platform on multiple robot bodies, tasks and learning algorithms using saved telemetry, and not by looking at demonstration footage alone.

1.4 Research questions

The objectives above describe what I built. The four questions set out below are what I intended to answer using the finished system, and they organise both the implementation and the evaluation that follow.

Table 1. Research questions used to organise the implementation and evaluation.

ID	Research question
RQ1	How can a no-code interface translate a URDF and a natural-language goal into a valid RL configuration while retaining human oversight?
RQ2	To what extent can morphology-derived configuration and expression rewards support different robot bodies without robot-specific application logic?
RQ3	How do PPO, SAC and TD3 differ across quadruped and humanoid locomotion experiments?
RQ4	How do parallel environment count and robot complexity affect GPU simulation throughput?

1.5 Scope and contributions

EasyRTG is a single-user desktop program that is designed to be used for simulated URDF robots. It provides an integrated workflow and a set of engineering mechanisms such as morphology inference, expression rewards, config auditing, agent tool use and run provenance; without a new reinforcement-learning algorithm. Physical deployment, terrain curricula, camera observations, ROS integration and multi-agent learning are all outside what I implemented. The results presented afterwards should therefore be read as evidence that the platform can configure and train several simulated robots. They are not evidence of sim-to-real transfer, and they are not evidence of universal no-code usability.

1.6 Organisation of this report

Chapter 2 covers the literature on which this work builds: the learners I used, the shift to GPU-native simulation that made the experiments feasible, the reward-design literature that explains why reward authoring is the real bottleneck, and the platforms closest to the accessibility goal of EasyRTG. Chapter 3 sets out the design-science method I followed, the evidence sources that influenced the design, the requirements I imposed on myself and the experimental protocol. The architecture and the three mechanisms that carry the contribution are described in Chapter 4. The history of development, including the parts that did not work, is detailed in Chapter 5. Chapter 6 presents the experiments together with the results from the run archive. Chapter 7 answers the research questions and states the limitations plainly. Chapter 8 concludes and spells out what I would do next.

2 Background and Literature Survey

2.1 Reinforcement learning for robot control

In reinforcement learning, a policy takes its inputs from observations and is optimised through the rewards collected during interaction with an environment [1]. The formulation has nice properties for robotics; the desired behaviour can be expressed without having to provide a labelled action for every state the robot could be in. The same properties make robotics challenging, though, since continuous action spaces, contact dynamics and long time horizons all increase instability as well as the cost of reward-design mistakes [2]. That is why learning is typically done in simulation before a policy is considered for real hardware, and that is why every experiment reported here was done in simulation.

2.2 The three learners: PPO, SAC and TD3

I employed three algorithms, and the reasons for choosing them influence how Chapter 6 should be read. PPO is an on-policy approach which restricts destructive updates through a clipped surrogate objective [3], and it has been widely adopted as the standard baseline for locomotion tasks because of its simplicity and its reliable implementations. SAC is off-policy, and maximises both expected return and policy entropy [4], which keeps exploration active and tends to help in continuous control. TD3 uses twin critics, delayed policy updates and target-policy smoothing to deal with value overestimation in deterministic actor-critic methods [5].

EasyRTG started with PPO only; SAC and TD3 were added late in the project, once the question had shifted. Originally I only wanted to know whether my pipeline would allow a robot to learn at all. By August the more interesting question was whether the different learners would expose different weaknesses in the same task and, as Section 6.8 shows, they did.

Table 2. Algorithm rationale and its observed role in EasyRTG.

Algorithm	Learning style	Reason for inclusion	Observed project role
PPO	On-policy, stochastic	Common locomotion baseline; stable implementation	Strong quadruped results; weak Valkyrie survival
SAC	Off-policy, maximum entropy	Exploration and sample reuse in continuous control	Best Go1 and Valkyrie policies
TD3	Off-policy, deterministic	Tests whether twin critics reduce overestimation	Strong Go1 result; Valkyrie later diverged

2.3 From single environments to massively parallel simulation

Gym standardised the interface between an agent and an environment [6] and Gymnasium and Stable-Baselines3 provided the maintained versions and APIs built on top of these that EasyRTG's first CPU lane was built on [7, 8]. PyBullet allowed me a quick way of loading a URDF, looking at the joints and checking my state calculations with a second implementation [9]. All three were suitable candidates for proving the correctness of an instance of the robot, but none of them could provide the amount of experience the experiments for locomotion needed, and it is this requirement that necessitated an architecture of the type outlined in Chapter 4.

2.4 Isaac Gym and the GPU-native turn

Makoviychuk et al. showed that physics simulation and policy learning can both be kept resident on the GPU, thus eliminating the CPU transfer bottleneck and boosting achievable throughput by orders of magnitude [10]. In the context of this project that is the most salient result, since it represents the difference between simulating one robot at a couple of hundred steps per second and four thousand robots at over thirty thousand steps per second. In practical terms it turns a one-time experiment into an experiment that can be repeated until it becomes informative. In Section 6.4, you can see where that difference landed on the hardware that I had.

2.5 Orbit and Isaac Lab

Mittal et al. introduced Orbit [11], a modular framework for robot-learning environments based on Isaac Sim, since developed into Isaac Lab. Orbit is relevant to this project for nearly the opposite of the reasons that Isaac Gym is relevant: it shows how much structure a well-designed environment framework can provide, and at the same time how firmly that structure is aimed at developers. Its environment definitions are Python classes, which is entirely reasonable for its intended users, and is also exactly the assumption EasyRTG is trying to remove. I use Isaac Lab as the training substrate [25], while keeping its configuration surface behind the desktop workflow.

2.6 Learning to walk in minutes

In the case of walking, Rudin et al. were able to train thousands of simulated ANYmal robots to walk in just a few minutes on a single GPU [12]. This paper set my expectations of what a legged-locomotion run should cost, and it is why ANYmal was the first robot I tried. It is also a useful corrective to my own work, since the speed they report comes from an environment, a reward and a curriculum tuned by domain experts for that particular robot. My own ANYmal runs took ten attempts before one of them reached 95 per cent of the episode cap (Appendix D). I think the distance between those two facts is a fair measure of what a general no-code pipeline currently costs against a specialist implementation.

2.7 Reward design as the real bottleneck

A reward is not a neutral description of a task. It is an optimisation target, and an agent will take advantage of any disparity between the intended behaviour and the quantity that is actually measured. It can be shown that potential-based shaping preserves an optimal policy under

certain conditions [13], but practical locomotion rewards are mixtures of objectives and penalties for which no such guarantee exists. Reproducibility-related challenges further complicate the picture: Henderson et al. and Engstrom et al. both demonstrate that implementation details and evaluation choices can make a significant difference to the results of deep RL [14, 15].

I use this body of work as justification for considering reward authoring as a correctness problem and not just as a configuration detail. This is why EasyRTG spends time compiling and auditing the reward expression before spending any GPU time, and why, throughout Chapter 6, episode length is reported alongside reward. Section 6.7 contains the concrete case that this literature predicts: a policy that scored highly on the galloping reward without ever galloping.

2.8 Eureka: language models writing rewards

On reward design itself, Ma et al. have shown with Eureka that a large language model can write reward code and then refine it using training feedback, reaching human-competitive reward design across a set of tasks [16]. Eureka is the closest published precedent for the assistant in EasyRTG, and the differences between the two are intentional. Eureka writes executable reward code and searches over it evolutionarily. My assistant writes expressions over an approved state vocabulary, and those expressions are compiled through an AST whitelist and audited on synthetic state ranges before they can be saved. I gave up expressive freedom in exchange for inspectability and safe tool use, and I come back to that trade in Section 7.2, since it is the one I feel least confident about.

2.9 DrEureka: pushing language-assisted design toward hardware

DrEureka continued this line of research towards sim-to-real transfer, using a language model to design both the reward and the domain-randomisation configurations [17]. I include it as an honest indicator of what my own project does not attempt: EasyRTG stops at the recorded simulated policy, with no domain randomisation and no hardware deployment. More specifically, DrEureka shows what the next stage of this work would have to look like, and Section 8.2 says as much.

2.10 General-purpose simulators: Gazebo and Unity ML-Agents

Gazebo is a general simulator for robots, equipped with programmatic interfaces and a strong ROS integration [18], and Unity ML-Agents turns a game-engine scene into a learning

environment [19]. Both are more capable than EasyRTG in their own field, and neither is trying to solve this problem. The advantage of Gazebo is how widely it is integrated in the robotics space, but it comes with a highly time-consuming setup effort, while the advantage of ML-Agents is the flexible integration of visual environments, which comes with an authoring effort that is specific to the engine. In both cases the route from a robot file to a trained policy passes through code that will not be accessible to a non-expert user.

2.11 Unity RL Playground: the closest accessibility claim

Of the published work, the closest approach to my accessibility goal is the Unity RL Playground by Ye et al., which provides one-click locomotion training for imported mobile robots [20]. It confirms that the gap I am chasing is real, and that others are chasing the same gap. EasyRTG differs in what it exposes rather than in what it hides: a shared state vocabulary that the user can author rewards against, an audit step that is performed prior to training, and a retained archive of runs, so that a claim can be reviewed after the fact. I did not find reported evidence that Unity RL Playground covers reward authoring at this depth, but I imagine that is because the scope they report on is narrower than what I have tried to compare against.

2.12 Where EasyRTG sits

Overall, this literature implies that the algorithms are there, the simulators are there and even language-based reward generation is there. What is still missing is orchestration, that is, making the robot structure, the semantics of rewards, the training controls and the supporting evidence all visible in one place to someone who does not start off as an expert. That is the gap EasyRTG was designed to test, and the value of the project lies in the concrete design together with a documented account of where the abstraction holds and where expert judgement is still needed.

Table 3. Positioning EasyRTG against representative robot-simulation and learning platforms.

Platform	Robot import	Training interface	Reward authoring	Primary trade-off
Gazebo/ROS	Rich model ecosystem	Programmatic	Controller/environment code	Broad robotics integration; substantial setup
Unity ML-Agents	Scene/component based	Unity + Python	Agent scripts and trainer config	Flexible visuals; engine-specific authoring
Isaac Lab	USD/URDF workflows	Python/configuration	Environment/reward code	High throughput; expert-oriented
Unity RL Playground	Imported mobile robots	One-click task workflow	Task framework	Closest accessibility goal; narrower reported scope

Platform	Robot import	Training interface	Reward authoring	Primary trade-off
EasyRTG	URDF with inferred morphology	Desktop no-code flow	Audited expressions from user or assistant	Accessible integration; currently simulation-only

3 Research Methodology and Requirements

3.1 Design-science approach

Since the research questions address the issue of how to design an integrated tool and of what happens when it is applied, I adopted a design-science approach where knowledge is developed in the process of building and testing an artefact [21]. A purely observational study could not have answered them. The process was deliberately iterative: I would identify a barrier, put in a mechanism I thought should overcome it, watch it fail, refine it, and then evaluate the new behaviour. The process outlined by Peffers et al., which involves problem identification, definition of objectives, design and development, demonstration, evaluation and communication, is the structure that I adopted [22], and Chapter 5 is the record of going round that loop.

3.2 Evidence sources

Design-science evaluation makes sense only if it is based upon evidence that can be examined by somebody else, so I limited myself to four sources. The Git history, 50 commits between 11 June and 14 August 2026, records the sequence of architectural changes. The source code and its automated tests establish what has actually been realised, rather than what was planned. Every Isaac run stores its own configuration and telemetry, so any number quoted in Chapter 6 can be recomputed from the archive instead of being taken on trust. Recorded policy videos supply the behavioural evidence that numbers on their own cannot deliver.

There is one rule that the analysis follows throughout. Only those timestamped directories that contain numeric reward telemetry are included, and the directories without it are counted as pre-telemetry failures rather than being silently dropped. That rule is the reason the run count in this report is higher than the count given in my presentation, and Section 6.1 explains the difference.

3.3 Functional and non-functional requirements

I held the implementation to eight requirements, each of which is tied to something checkable in the repository rather than to a general aspiration.

Table 4. Traceable project requirements.

ID	Requirement	Verification evidence
FR1	Load and inspect an arbitrary URDF robot	Robot panel, morphology tests, A1/ANYmal/Go1/Valkyrie
FR2	Create observations, actions, rewards and terminations without editing code	Config panels and expression editor
FR3	Let an assistant propose and apply configuration through tools	Agent toolbox and confirmation gates
FR4	Launch and monitor GPU-parallel training	Isaac worker and telemetry charts
FR5	Replay and compare completed policies	Evaluation worker, videos and run archive
NFR1	Reject invalid or unsafe reward expressions before launch	Compiler, audit and configuration tests
NFR2	Keep the desktop responsive during long-running work	Subprocess workers and polling/streaming
NFR3	Preserve run provenance	Saved config, telemetry, checkpoints and video

3.4 Experimental protocol and metrics

The run archive spans experiments from 13 July to 18 August 2026 (all of them were run on a single RTX 5000). This set comprises 34 usable runs: 8 A1 PPO, 10 ANYmal PPO, 4 Go1 PPO, 3 Go1 SAC, 2 Go1 TD3, 3 Valkyrie PPO, 3 Valkyrie SAC and 1 Valkyrie TD3. This is not a balanced distribution, and I do not present it as one. Every comparison in Chapter 6 is described as an observed project result, and not as a statistically complete benchmark.

Forward velocity, uprightness, lateral drift and flight phase are the task-specific measures that I use, alongside mean episode reward, mean episode length and environment steps per second. Episode length is always reported as a fraction of the 1,000-step limit, because that fraction answers the question reward cannot: did the robot stay upright? When computing fleet summaries, the run with the highest observed reward is used for each robot and algorithm combination. This makes the summary tables easier to read, but it also introduces best-run selection bias, which I treat explicitly instead of implicitly in Section 7.3.

3.5 Software verification

Aside from the training experiments I also ran the repository's backend test command in a new, temporary directory with third-party pytest plugin autoloading disabled. There were 256 passes, 1 skip and no failures. The tests cover configuration stages, expression compilation, morphology, contact state, episode timing, Isaac bridge calculations, action authoring, spawn height and smoke behaviour. I could not get the Flutter command-line tool to return in the environment available to me, so for now I make no fresh claim about the status of the frontend analysis; that gap is listed in Section 7.3 with the rest.

4 System Design and Architecture

4.1 Architectural overview

These requirements, defined in Section 3.3, actually pull in two directions at once, in that FR4 wants thousands of parallel environments running on a GPU while NFR2 wants a desktop that stays responsive while they run. EasyRTG resolves that tension with a desktop client and a local orchestration service. The only user interface is offered by Flutter [26]. FastAPI is responsible for the application state, provides REST and streaming endpoints, supervises workers and keeps a record of project state [27]. The LLM assistant has no privileged path into the system; it calls the same bounded tools that the application itself uses. Expensive simulation is split into two lanes, so that a lightweight check never consumes a GPU training session.

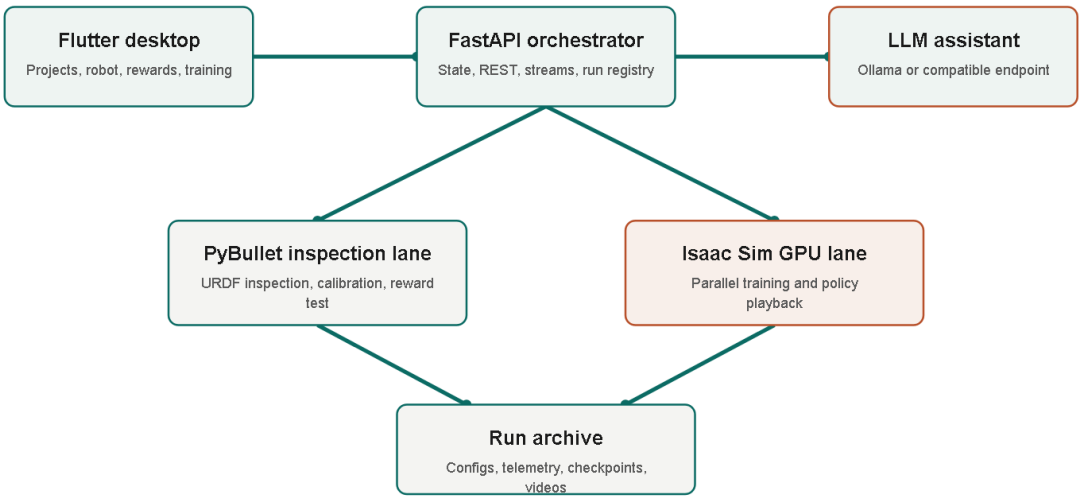


Figure 2. EasyRTG architecture, and the separation between the inspection and GPU training lanes.

The three sections that follow describe the mechanisms that carry the contribution of the project, in the order in which a user meets them. It is only sensible for the assistant to take part once the structure of the robot has been established and a reward has been formulated against that structure, so the robot is understood first, then the reward is written, and only then is the assistant allowed to help.

4.2 URDF inspection and morphology

My first prototype came up with the robot roles from the link and joint names, looking for parts like 'hip', 'foot' and 'ankle'. This had been successful with the examples I had, but it fell apart the moment those naming conventions changed. That failure is the source of the morphology layer, which considers the URDF as a kinematic tree rather than as a list of strings. In that layer it identifies the base link, the movable joints, leaf structures, candidate limbs and contact links according to the connectivity and the joint properties. Standing height and spawn height are measured relative to the robot itself, and not simply copied from whichever sample robot I tested last. This lets dimensionless signals such as `height_ratio` and `upright` mean the same thing on a quadruped as on a humanoid.

Not all URDFs will automatically be trainable. Poor inertial values, unrealistic joint limits and ambiguous contact geometry all still need calibration, and this is mentioned as a limitation in Section 7.3. What the layer does achieve is the following: a new robot starts from structural evidence, and not from a list of names that was written for a previous robot.

4.3 Expression rewards and the shared state vocabulary

Once the structure of the robot has been derived, the problem becomes how the user is to instruct the robot in its behaviour. A fixed reward catalogue, in which users could only pick components that I already knew to be good, seemed to be the safe route to take. Ultimately it did the exact opposite of what I wanted: it limited the range of behaviours the system could express, and it pushed every new task back into the application code. Each reward term is now represented as a short mathematical expression over a state vocabulary: `forward_velocity`, `upright`, `height_ratio`, `mean(contact)`, `flight_phase` and `episode_progress`. The morphology layer supplies the vector widths, so `mean(contact)` means the same thing whether the robot has two contact limbs, four or six.

Safety is provided by the compiler, and not by restricting the menu. Expressions are parsed into an abstract syntax tree and are accepted only if every node in that tree is on a whitelist. Attribute access and subscripting are excluded, which is what prevents the arbitrary-Python reward path I deleted from creeping back in. Configuration patches compile every enabled expression before saving, and an audit then evaluates the reward scale across synthetic state ranges, flagging non-finite values, dominant penalties, all-penalty designs and sums that depend on the number of degrees of freedom. None of this ensures that a reward will yield good learning. What it does is catch several expensive classes of failure before any GPU time is committed, which on a 40-million-step run is worth several hours.

4.4 Assistant and human oversight

An expression layer on its own is not going to be useful to somebody who would not write those expressions themselves, and the assistant is what closes that gap. It can read robot details, calibrate a goal, patch a configuration, test a reward, start or stop training and inspect past runs. Basic instructions can be routed to a local Ollama model, while more sophisticated goal interpretation can use an OpenAI-compatible endpoint. The system prompt carries the robot-specific vocabulary along with the available reward priors.

Also important is that the tools which change configuration or control runs stay gated, so that the user approves anything consequential in advance. I did not create this pattern myself; it follows established guidance for human-AI interaction, which is to communicate what the system can do, make changes visible, and preserve correction or undo [23]. It was also the design decision that most often slowed me down while using my own application, which I take as an indicator that it is doing its job.

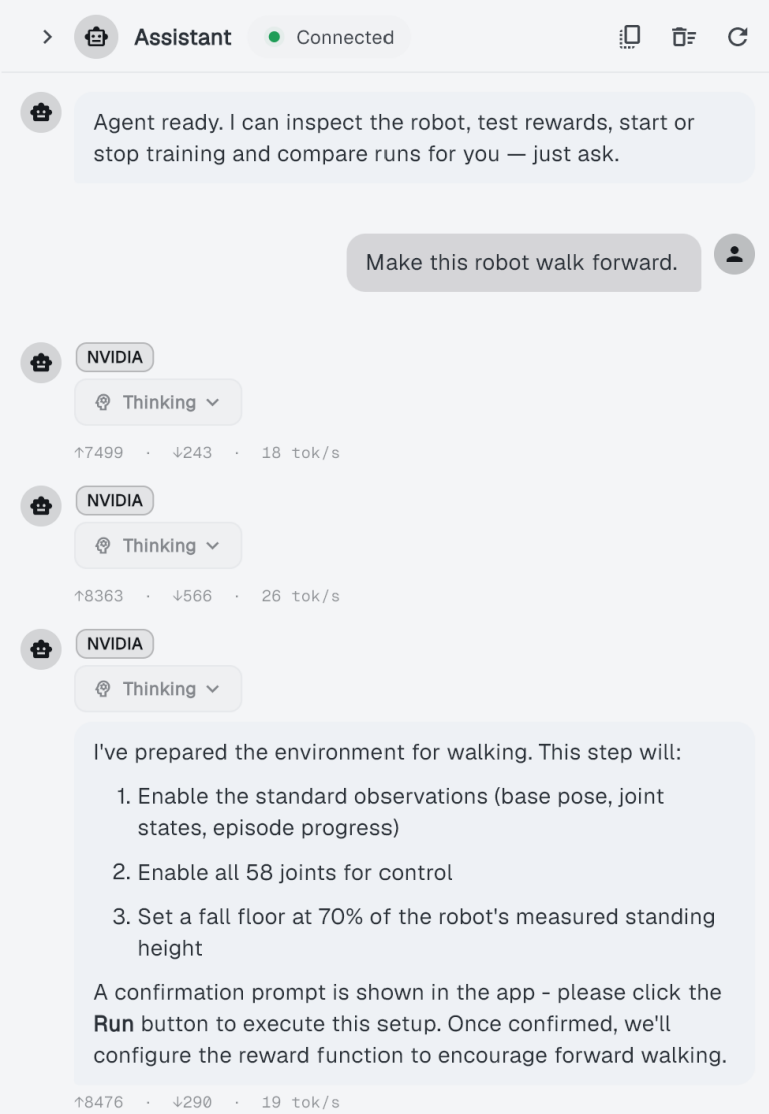


Figure 3. The EasyRTG desktop assistant, showing a generated training configuration and the changes offered for review.

4.5 Training, monitoring and evaluation

Once a configuration is approved, Isaac training runs in a separate subprocess under Isaac Sim's own Python environment, following the installed Isaac Lab workflow [25]. The worker writes telemetry to a file, and the backend tails that file and pushes the updates to the Flutter charts. Each run keeps its exact environment configuration, along with its observation and action contracts, checkpoints, log and evaluation video. Live evaluation can launch NVIDIA's WebRTC client. Recorded evaluation runs headless and produces an MP4 file, which stays replayable inside the desktop application.

That retained evidence is not bookkeeping for its own sake. A reward curve on its own cannot determine whether a humanoid is walking forwards, drifting sideways or merely staying upright without completing the task. Several of the interpretations described in Chapter 6 were

only possible because the configuration and the video of each run had been kept and could be inspected afterwards.

5 Implementation and Iterative Development

5.1 From proposal to working system

As stated in my original proposal, I would be creating a six-week CPU-first application with predefined reward parts and optional custom Python, that would work on an average laptop, and would be built on PyBullet, Gymnasium and PPO from Stable-Baselines3. The finished system differs from that plan in almost every part of it that carried risk. Training moved to Isaac Sim on an NVIDIA GPU. Custom Python was removed completely. Reward components became audited expressions, and the use of joint-name rules was replaced by morphology inference. The evaluation, which was kicked off with PPO on sample robots, was expanded to three algorithms across four robot families.

I do not think that record indicates that the proposal was poor. It shows that the prototype was used for what a prototype is for, which is gauging what the project really needs. The remaining sections of this chapter set out the specific sequence of failures that produced each of those changes.

5.2 Milestone history



Figure 4. Major implementation and experiment milestones, reconstructed from Git dates and the run archive.

Firstly, the commits dated 11 June, titled 'initial 300x300 30fps stream' and 'most of the app is working and is stable', created a visible prototype. On 14 June, 'most of the agent is working' and 'Make Windows build/run work; stream JPEG sim frames' joined the assistant to a desktop build. These messages are informal, but the thinking behind them was to make one complete interaction visible before refining anything underneath it.

The next phase was more indirect. 'Add MJX backend, config revisions, and agent routing', on 29 June, was a trip to try an alternative high-performance backend, and the next day's commit, titled 'switching to linux as mujoco isnt working good on win', records why that attempt stalled. Although the branch never became the definitive training lane, it did prove one thing to me that a nice front-end interface can never paper over: simulator and operating-system constraints do not go away because you have hidden them behind a pretty interface. EasyRTG then used Isaac Sim as a separate GPU subprocess, and this way the main application remained cross-platform, while the training lane could be as platform-specific as necessary.

On 27 July the locomotion case study was started properly with 'added unitree go1 robot as casestudy'. My walking reward quickly discovered an easier way to travel than walking, which was to stay low and crawl. That failure became a robot-relative height termination in the commit 'fix: reject crawling in walk goals'. On 28 July, 'Derive robot structure from URDF, not joint names; add expression layer' eliminated another source of hidden robot-specific assumptions.

The most concentrated redesign was on 29 July. Four commits made huge changes to what the application was: 'Evaluate rewards as expressions on the Isaac lane', 'Give the Isaac lane real contact state, so a flight phase is expressible', 'Let the agent author the reward; delete the 4-keyword goal enum', and 'Delete the reward catalogue: expressions are the only reward path'. The same day ended with 'Record the gallop result; gait is no longer inexpressible'. That result was far more important than the commit message indicates, because galloping cannot be described by forward velocity alone: it requires all four feet to leave the ground at the same time. Expressing it needed contact state in the Isaac lane and a reward language capable of describing a flight phase. Until both of those existed, the gait was not slow to learn, it was unsayable.

Work after this point made changes to the interface and the error handling. The commit 'Stop expressions written against observation names from being saved', dated 30 July, was born out of an authoring mistake that I made myself. The process of interface refinement was ongoing until 7 August, when it reached 'most stable and clean ui'. The commits of 14 August reorganised the architecture notes and updated the presentation and the README files, and the run directories added on 18 August expanded the analysable dataset beyond the numbers given in my earlier slides.

5.3 Key engineering decisions

Six decisions survived that sequence. Each is listed below alongside the alternative I actually encountered, since in every case I attempted the alternative first.

Table 5. Engineering decisions that emerged through iteration.

Decision	Alternative encountered	Reason retained
Two simulation lanes	Use PyBullet for everything	Fast inspection stays lightweight; Isaac provides training throughput
Expression rewards	Fixed catalogue or arbitrary Python	More behaviours than a catalogue; safer and more portable than exec
URDF-derived morphology	Joint-name heuristics	Supports unfamiliar naming and robot-relative measurements
Subprocess Isaac worker	Embed Isaac in FastAPI	Isaac uses its own Python and runtime, and can fail without taking the UI down
Human approval gates	Fully autonomous tool execution	Makes consequential changes visible and recoverable
Saved run directories	Ephemeral charts	Enables later recomputation, comparison and failure diagnosis

5.4 Security and reliability improvements

One consequence of making a no-code interface is that the user becomes more dependent on the protections built into the application, which makes those protections part of the contribution rather than housekeeping. The following insecure behaviours were identified in an internal assessment: unsafe pickle loading, arbitrary custom reward execution, ungated agent changes and permissive CORS settings. I worked my way through the high-priority issues in July; eventually, I got rid of the custom-Python reward path altogether, rather than defending it with a denylist that would have had to be expanded over and over, so that expression compilation now excludes the dangerous syntax by construction. The training, tuning and evaluation workers also look for conflicting jobs, and the desktop backend launcher will not terminate a server if it was not the one that launched it.

6 Experiments, Results and Findings

6.1 Run coverage and failure history

The number of runs has to be stated before any result is given. My presentation summarised 30 runs up to 3 August 2026, and the README later summarised 32. When the final archive was re-scanned for this report, 40 timestamped directories were found, of which 34 contained numeric reward telemetry and six contained none. I report the changing count rather than hiding

it, because it is an accurate record of a project whose final fortnight added most of its algorithm coverage. Each of the fleet-level numbers below is based on the 34 runs that were amenable to analysis, and they are detailed individually in Appendix D.

Twelve of those runs held at least 95% of the episode cap. Several others achieved only partial stability, and one Valkyrie TD3 run did provide usable telemetry before it was lost to a numeric failure. The chronological picture, given in Appendix C, makes it clear that the shape of the project emerges from repeated problems of spawning, of termination, of reward scale and of learned behaviour: the successful policies only appear after those failures. A polished final video is an incomplete picture of development, and that chronological figure is what shows why.

6.2 Fleet-level results

Table 6. Best observed run for each robot and algorithm pairing.

Robot	Algorithm	Best reward	Episode length	% of cap	Peak steps/s
A1	PPO	4,767.0	967.5	96.8%	32,056.8
ANYmal	PPO	769.8	954.6	95.5%	21,055.5
Go1	PPO	2,073.9	962.7	96.3%	27,500.0
Go1	SAC	2,250.1	1000.0	100.0%	27,840.3
Go1	TD3	2,112.7	990.9	99.1%	28,975.6
Valkyrie	PPO	276.1	204.2	20.4%	14,280.0
Valkyrie	SAC	1,283.8	821.0	82.1%	11,171.6
Valkyrie	TD3	287.0	206.2	20.6%	14,388.8

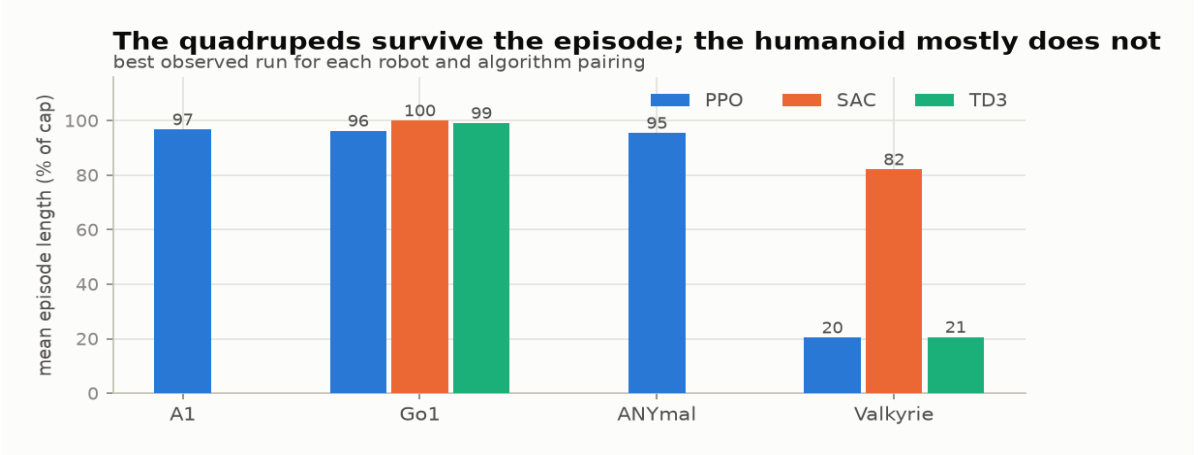


Figure 5. Episode survival for the best observed run in each robot and algorithm pairing.

A1 under PPO reached 967.5 steps together with the highest reward recorded anywhere in the archive, 4,767.0, while ANYmal under PPO achieved 954.6 steps. Go1 reached 962.7 steps

under PPO, the full 1,000 under SAC and 990.9 under TD3. The learners were divided on Valkyrie, with PPO finishing at 204.2 steps, SAC at 821.0 and TD3 at 206.2.

However there is one caution which needs to be added regarding the interpretation of the table. The reward function is different across robots and tasks, hence the reward values must not be compared across rows as though they were a shared score. A1's 4,767.0 does not mean that A1 learned better than Go1; it means that A1's reward expression pays more per step. The `% of cap` column is the only column in the table that is directly comparable across rows, which is exactly why I report it everywhere.

6.3 What the reward actually paid for

Notably, because reward totals are not comparable from robot to robot, it is more useful to ask what a given policy was actually being paid for. EasyRTG logs each reward term separately, which makes it possible to decompose the total at the point at which a policy performed best.

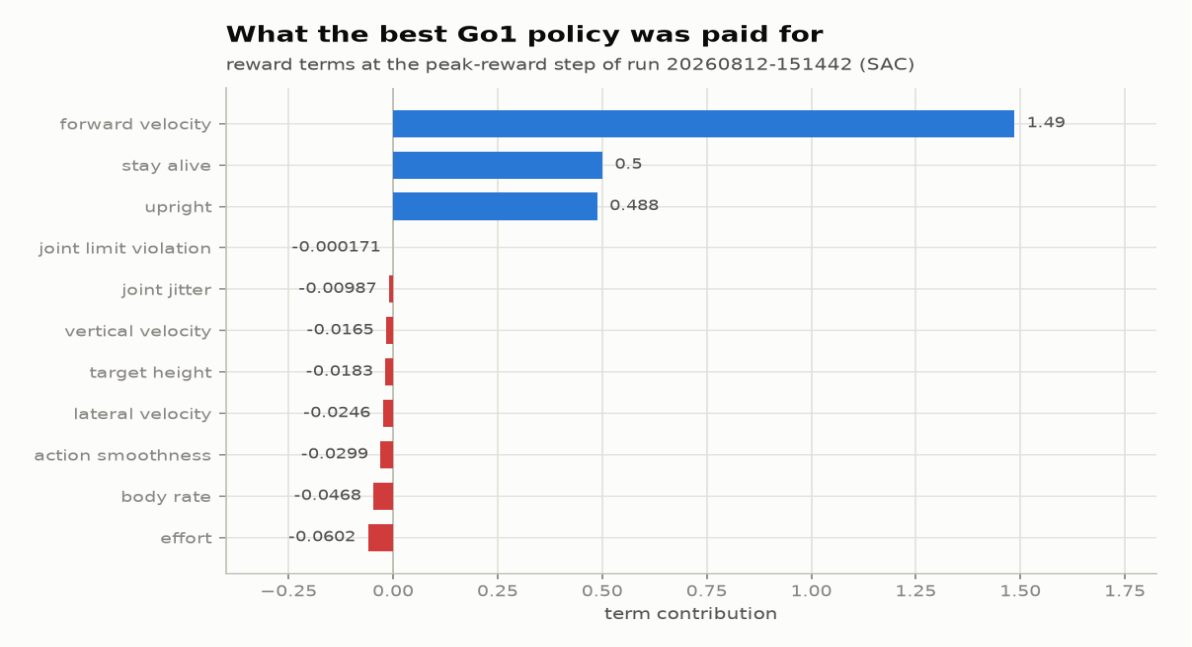


Figure 6. Term-by-term decomposition of the reward at the peak-reward step of the best Go1 run.

Only three positive terms dominate in the decomposition, the forward velocity at 1.49, staying alive at 0.50 and uprightness at 0.49, against penalties that are between one and four orders of magnitude smaller. The biggest penalty is effort at -0.060, while joint-limit violation, at -0.00017, is effectively zero, which tells me that the policy was not fighting its joint limits at all.

The decomposition is not only a useful sanity check but also a warning. It verifies that the policy earned its score by moving forwards while upright, and not by gaming the penalty

structure. At the same time it demonstrates how little the penalties actually constrained it. A reward as dominated by its positive terms as this one leaves room for just the sort of misspecification documented in Section 6.7, and it is the strongest argument I have for keeping the audit step described in Section 4.3.

6.4 Throughput and the GPU lane

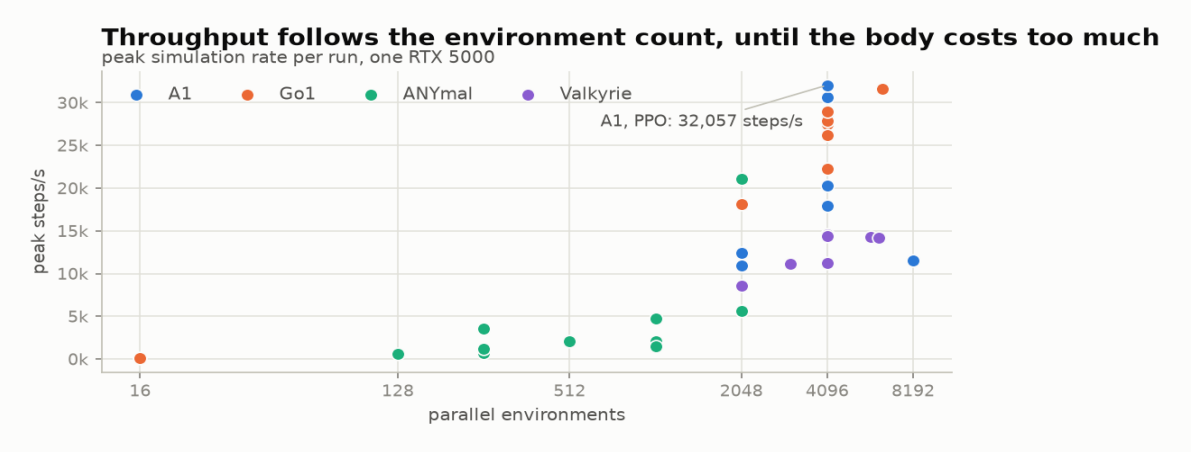


Figure 7. Peak simulation throughput against the number of parallel environments, coloured by robot.

At 4,096 environments, the maximum throughput I measured was 32,056.8 environment steps per second, on A1. Go1 also managed recorded runs of over 30,000 steps per second, while Valkyrie never got past roughly 14,600 steps per second at any batch size I tried.

Raising the number of simulated environments increases the parallel work available, but the cost of each simulated step is set by the morphology, and a humanoid with more joints and more bodies is not a fair throughput comparison with a quadruped. Hence RQ4 cannot be reduced to a single scaling rule, but rather to an interplay between two of them: parallelism sets the opportunity, and the complexity of the robot sets the practical ceiling. The two runs at 16 environments, added on 18 August, make the lower bound of that relationship concrete, since both fall below 100 steps per second, which is roughly three hundred times slower than the same robots at 4,096 environments.

6.5 How long learning took

Throughput is only one half of what makes an experiment affordable; the other half is the number of steps the policy needs. I measured this as the number of environment steps before the smoothed mean episode length first holds above 900, which is the point at which a policy has stopped falling over rather than merely got lucky in one logging interval.

Strikingly, exactly twelve runs cross that threshold, and they are the same twelve that reach 95% of the cap in Section 6.1 as well, which is a consistency check worth noting because the two quantities are calculated by different routes. The complete ordering is provided in Appendix C. The spread is wide: the best ANYmal run crosses the threshold after 2.51M steps, while the slowest Go1 run takes 17.60M steps to cross it. It is also worth noting that the fastest crossings are not the highest-scoring runs. A1's best-reward run (20260728-202311, reward 4,767.0) took 16.42M steps to stabilise, while an earlier A1 run stabilised after 4.03M steps and scored less than half as much. Stabilising early and scoring highly are not the same achievement, and averaging the two into a single performance figure would have hidden that.

6.6 Go1: three learners, one solved robot

I noticed that only Go1 was used at the same environment count with all three learners, so this is the one I'm using to show comparisons between all three.

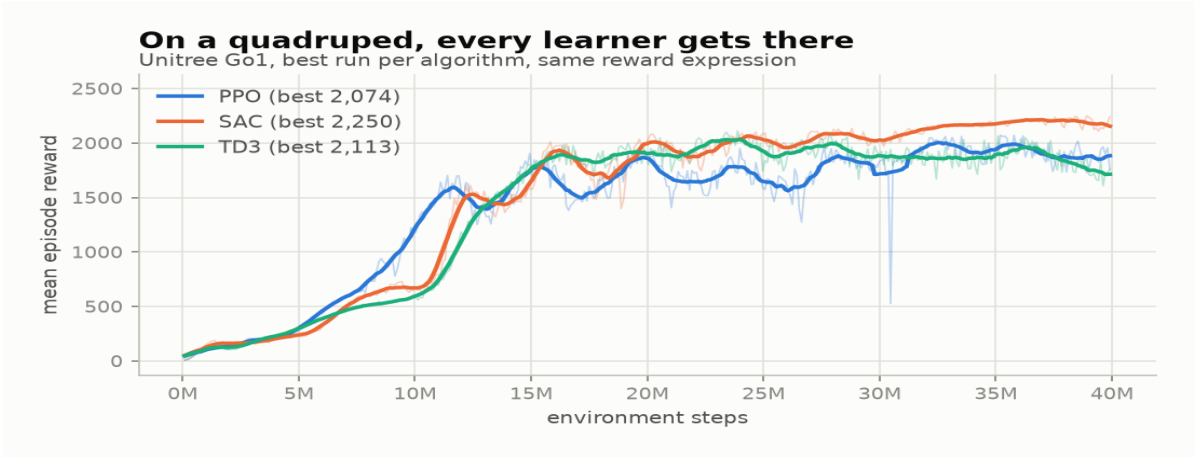


Figure 8. Reward for the best Go1 run under each learner, at 4,096 environments with the same reward expression.

The difference between the different learners is not significant on a robot where the task is well solved. All three are within 8% of each other on the reward metric, with SAC at 2,250.1, TD3 at 2,112.7 and PPO at 2,073.9, and all three hold essentially the full episode, at 100%, 99.1% and 96.3% of the cap respectively. The key difference between the traces is in their shape rather than their destination: SAC climbs in distinct steps, PPO climbs more smoothly, and TD3 is fastest at the beginning and then flattens.

The present result does not imply that these three algorithms are equal. It shows that this task, on this robot, is easy enough that all three of them saturate it. The separating comparison is thus not here but on the humanoid, where the task is challenging enough to separate them, and that comparison is made in Section 6.8. What the Go1 experiments do show is the necessary

baseline: when the problem is tractable, my pipeline does not favour one learner, so any separation seen on a harder problem is more attributable to the problem than to my implementation.

6.7 Go1 walking and galloping

Go1 also produced the clearest demonstration in the whole project that the behaviour is defined by the reward. Walking rewarded forward motion, uprightness and keeping the feet on the ground, whereas galloping required a flight phase with all four feet off the ground. In the presentation experiments the walking policy travelled at 0.98 m/s, held an uprightness of 0.99 and survived 963 steps, while the galloping policy reached 1.38 m/s, survived the full episode and was airborne for 43% of the stride. The robot and the training budget were the same in both experiments; the reward semantics were the only difference.

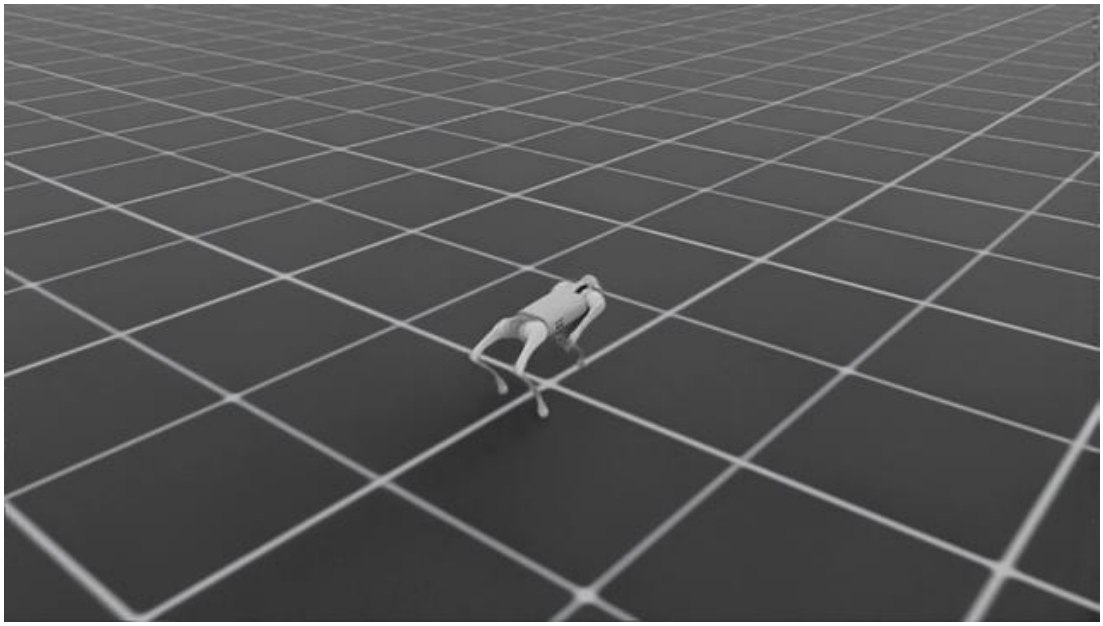


Figure 9. The Unitree Go1 during recorded policy evaluation.

The three-algorithm comparison then added a qualification that I did not want, but should have anticipated. TD3 produced the highest velocity achieved on the galloping task, of around 1.65 m/s, and yet it had the lowest flight fraction at 29%, against SAC's 52% at a slower 1.52 m/s. TD3 had learned a fast trot that scored well on my galloping reward without galloping at all.

This is exactly the failure mode that the literature on reward design predicts [13, 14], and finding it in my own system was in fact one of the more beneficial results of the project. It is why contact-state measurements and video inspection stayed in the pipeline next to reward, and it is why nowhere in this chapter do I treat a high reward as a finished result.

6.8 Valkyrie: a controlled algorithm comparison

The strongest comparison the archive supports is between SAC and TD3 on Valkyrie, where the reward expression is identical and the budget is the same 40M steps. This comparison is of greater value than the rest of the matrix, because everything except the learner is held constant.

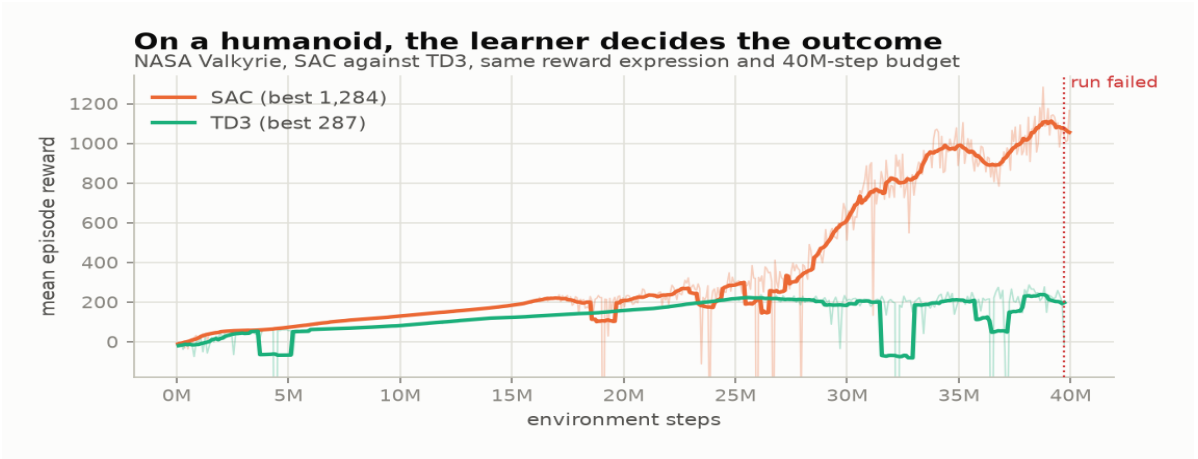


Figure 10. Reward for Valkyrie under SAC and TD3, same reward expression and same 40M-step budget.

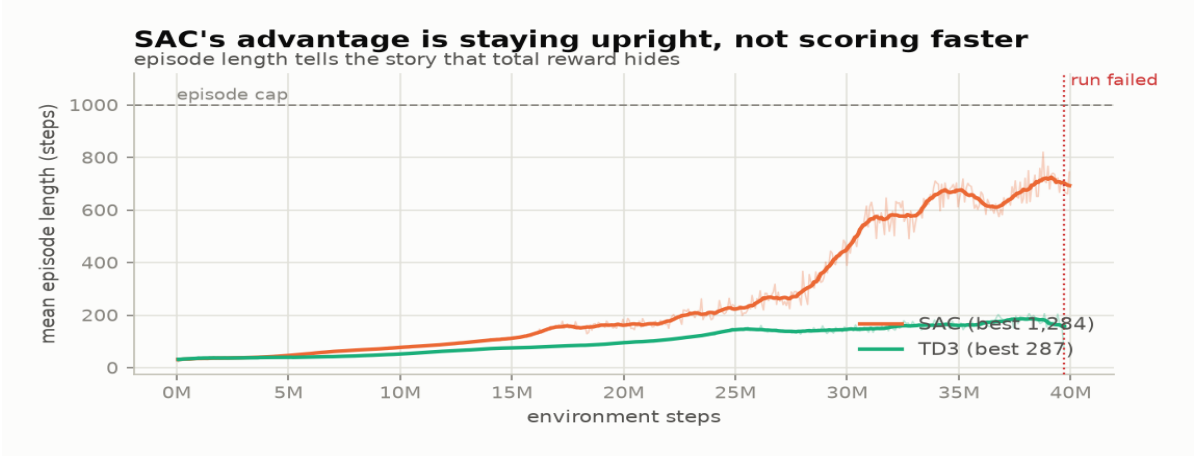


Figure 11. Episode length reveals that SAC's advantage came from staying upright, not from scoring faster.

Initially both learners made progress, and up to about 20 million steps they are hard to tell apart. Then they diverge. SAC reached a best mean reward of 1,283.8, with a mean episode length of 821.0 steps. TD3 flattened at around 200 steps of survival, finished with a best reward of 287.0, and eventually stopped altogether after NaN observations; the dotted line in the figures marks where that run died.

The reason this comparison changes the interpretation of reward is in the second figure. At their best checkpoints the two policies received broadly similar reward per step. SAC's total was larger because it accumulated those terms for about four times as many steps before falling, so its advantage was survival and not scoring rate. The TD3 telemetry also showed reward terms

exploding by many orders of magnitude as failure approached, which suggests critic and numerical instability rather than a simple failure to move forward.



Figure 12. The NASA Valkyrie during recorded SAC policy evaluation.

Valkyrie is therefore the humanoid answer to RQ3, and it is a narrow one. The choice of learner materially changed the outcome on the hard task, in favour of SAC, under the one configuration that I tested. This should not be interpreted as an endorsement of SAC in general, and Section 7.3 explains why I cannot make that claim from this archive.

6.9 Functional verification

The second form of evidence contained in this report is the 256 passing backend tests. They do not show that any robot has learned anything. What they verify is the machinery that makes the training results interpretable at all: morphology, action mapping, the state vocabulary, expression guards, episode clocks, contact signals, the Isaac bridge mathematics, configuration phases and failure handling.

Looking at the work for myself, I regard the combination as stronger than either half on its own: a visually convincing gait cannot prove that the configuration is safe, and a passing unit test cannot prove that the behaviour is useful. The galloping result in Section 6.7 is precisely a case where the tests were all green and the behaviour was still wrong, which is the most obvious argument I can give for retaining both kinds of evidence.

7 Discussion

7.1 Answering the research questions

RQ1 aimed at assessing how a no-code interface could turn a URDF and a given goal into a valid training configuration while keeping human supervision. The implemented answer is the staged workflow of Chapter 4: structural inspection, robot-relative calibration, reward authoring restricted to an approved shared vocabulary, compile-and-audit checks, and explicit confirmation for consequential tools. It is all done by the application, without the user touching any of the environment's source code. What I cannot tell you is how much expertise or how much time the workflow removes, because I did not run a comparative user study. That is the largest hole in the evaluation, and I expand upon it in Section 8.2.

RQ2 was on generalisation across robot bodies. The core pipeline has been able to produce runs that were amenable to analysis on all four bodies: the quadrupeds A1, ANYmal and Go1, and the humanoid Valkyrie. Structural morphology and dimensionless state terms further eliminated several hard-coded assumptions along the way. It looks promising with respect to cross-robot applicability within legged URDF models. It says nothing about manipulators, wheeled robots or closed-chain mechanisms, none of which I tested.

RQ3 posed the question of how the learners differ, and there were two parts to the answer, one of which makes little sense without the other. On Go1, where the task was tractable, all three learners achieved saturation and the choice barely mattered, as is shown in Section 6.6. On Valkyrie, where the task was not tractable, SAC was the only learner that demonstrated sustained walking, as described in Section 6.8. The finding is therefore about the interaction rather than about a ranking: the choice of learner becomes material exactly when the task is hard enough to expose it. I cannot go any further than that, because the algorithm coverage is unbalanced, the hyperparameters were not exhaustively tuned, and the summaries select the best observed run.

RQ4's question was related to throughput. The archive shows a clear relationship between the number of parallel environments and simulation speed, bounded by the cost of the robot being simulated: A1 reached 32,056.8 steps per second at 4,096 environments, while Valkyrie never approached that ceiling. This follows the literature on GPU-native simulation [10, 12] and, in hindsight, retrospectively justifies the architectural change to Isaac discussed in Section 5.2.

7.2 What the project learned about abstraction

The most difficult design problem in this project was deciding what to hide from the user. Hiding the simulator installation does not make its hardware and operating-system constraints go away, as I discovered with the MJX detour mentioned in Section 5.2. Hiding rewards behind a catalogue gives a simple interface and an inflexible platform, while allowing arbitrary Python gives a flexible platform and an unsafe one. EasyRTG's compromise is to expose only those decisions that actually affect learning, namely observations, actions, expressions, terminations and the choice of algorithm, while hiding the boilerplate needed to execute them.

I am least confident about where that line falls for the assistant. It is not an oracle, and I have never presented it as one; it is an authoring and orchestration layer sitting inside a system that has validation and human review around it. The six pre-telemetry failures and the many short-lived policies are part of the result rather than noise around it. Language assistance can reduce the effort spent on configuration. It does not remove the need for measurable checks, and it did not remove the need for me to diagnose the Valkyrie TD3 divergence by hand.

7.3 Limitations and threats to validity

- The experiment matrix is unbalanced. SAC and TD3 were concentrated on Go1 and Valkyrie, while A1 and ANYmal were run with PPO only, so no cross-robot algorithm claim is available.
- With fleet summaries, the highest reward for each combination is the one taken into account. These are hence best observed performance runs, and not averages over different random seeds.
- All throughput values reported here come from one single RTX 5000, and should not be used to make any assumptions about other GPUs without measurement.
- Every policy was evaluated in simulation. Domain randomisation and transfer to a physical robot were not attempted.
- The no-code claim in this report is architectural rather than empirical: no study involving end users was performed to measure task completion time, error rate, learning curve or perceived usability.
- Training quality depends on the URDF dynamics, and automatic morphology inference cannot repair unrealistic mass, inertia, collision or joint-limit data.

- Observations remain a fixed seven-key catalogue, even though rewards are expressions. Camera observations, terrain, velocity commands, ROS and multi-agent RL are not implemented.
- The fresh verification covered the backend alone. The Flutter command-line tool was stuck in the available environment, which means the current frontend lint and test status is unconfirmed and should be rechecked before submission.

7.4 Practical and academic significance

Practically, I believe EasyRTG shortens the path from a robot file to a recorded policy in a very tangible way, since configuration, training and the supporting evidence are all bagged up into one application. On the academic side, it offers a case study in how a language model can assist reward design without being handed unrestricted code execution, and it supplies a heterogeneous run archive which shows why reward, episode survival and task-specific behaviour have to be read together rather than separately.

Its most defensible contribution is not that robot learning has become effortless, because it is nothing as easy as that. It is that the hard choices have been made visible, reviewable and repeatable, and that when one of them was wrong, as with the galloping reward, the system made the mistake findable.

8 Conclusion and Future Work

8.1 Conclusion

EasyRTG's original concept was the application I would have wanted before my earlier robot project: load a robot, state a goal, and follow one process that I could actually understand. The realised implementation turns that into a working desktop platform with URDF inspection, morphology-derived configuration, assistant-authored reward expressions, GPU-parallel training, live telemetry and retained policy evaluation.

The supporting evidence covers 34 analysable Isaac runs across four robot families and three algorithms, a peak of 32,056.8 environment steps per second, strong quadruped locomotion, and a clear SAC advantage on the one Valkyrie task that was tested. However, it was the failures that taught me most, because they are what changed the system. Crawling produced robot-relative termination. Joint-name failures produced structural morphology. An inexpressible gallop produced contact state and expression rewards. The risky flexibility of arbitrary Python

produced a restricted expression compiler. And a galloping policy that never left the ground is why episode length and contact measurements sit beside reward throughout Chapter 6.

This means that the project as a whole is better described as a story of refinement than as an implementation of its proposal. The proposal was a plan for six weeks of CPU training. What it became is a platform whose central claim, that the robot structure, the reward semantics and the training evidence can all be made visible in one place to a non-expert, is now supported by working software and by a run archive that anybody can recompute.

8.2 Future work

The first priority is the study I did not run. A small but properly designed usability comparison between the EasyRTG workflow and a documented code-based baseline, measuring task completion time, configuration errors, requests for assistance and a validated usability scale, would allow the no-code claim to be evaluated rather than inferred. I regard this as more important than anything else proposed in this section.

On the technical side, observation expressions and per-episode velocity commands should be introduced together, so that one policy can respond to changing instructions rather than to a single fixed goal. A campaign worker could then propose, audit and probe reward revisions automatically, while still requiring the user's approval, which would be a natural extension of Section 4.4 rather than a replacement for it.

Beyond that, the work needs terrain and sensor observations, better automatic dynamics checks, more robots evaluated with balanced seeds and algorithm budgets, and measurement on more than one GPU. The hyperparameter choices used here were arrived at by iterating on individual experiments, and a reproducible study using a framework such as Optuna [24] would put them on firmer ground. Finally, EasyRTG would require both domain randomisation and export to real hardware before it could claim a full route from simulation to real robot behaviour, and DrEureka [17] provides a decent model of what that stage should involve.

Bibliography

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[2] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," International Journal of Robotics Research, vol. 32, no. 11, pp. 1238-1274, 2013.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in Proceedings of ICML, pp. 1861-1870, 2018.

[5] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in Proceedings of ICML, pp. 1587-1596, 2018.

[6] G. Brockman et al., "OpenAI Gym," arXiv:1606.01540, 2016.

[7] M. Towers et al., "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[8] A. Raffin et al., "Stable-Baselines3: Reliable reinforcement learning implementations," Journal of Machine Learning Research, vol. 22, no. 268, pp. 1-8, 2021.

[9] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016-2021. Available: https://pybullet.org/.

[10] V. Makoviychuk et al., "Isaac Gym: High performance GPU-based physics simulation for robot learning," arXiv:2108.10470, 2021.

[11] M. Mittal et al., "Orbit: A unified simulation framework for interactive robot learning environments," IEEE Robotics and Automation Letters, vol. 8, no. 6, pp. 3740-3747, 2023.

[12] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in Proceedings of CoRL, PMLR vol. 164, pp. 91-100, 2022.

[13] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in Proceedings of ICML, pp. 278-287, 1999.

[14] P. Henderson et al., "Deep reinforcement learning that matters," in Proceedings of AAAI, vol. 32, no. 1, 2018.

[15] L. Engstrom et al., "Implementation matters in deep policy gradients: A case study on PPO and TRPO," in Proceedings of ICLR, 2020.

[16] Y. J. Ma et al., "Eureka: Human-level reward design via coding large language models," arXiv:2310.12931, 2023.

[17] Y. J. Ma et al., "DrEureka: Language model guided sim-to-real transfer," in Proceedings of Robotics: Science and Systems, 2024.

[18] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in Proceedings of IEEE/RSJ IROS, pp. 2149-2154, 2004.

[19] A. Juliani et al., "Unity: A general platform for intelligent agents," arXiv:1809.02627, 2020.

[20] L. Ye et al., "Unity RL Playground: A versatile reinforcement learning framework for mobile robots," arXiv:2503.05146, 2025.

[21] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Quarterly, vol. 28, no. 1, pp. 75-105, 2004.

[22] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," Journal of Management Information Systems, vol. 24, no. 3, pp. 45-77, 2007.

[23] S. Amershi et al., "Guidelines for human-AI interaction," in Proceedings of CHI, paper 3, pp. 1-13, 2019.

[24] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proceedings of KDD, pp. 2623-2631, 2019.

[25] NVIDIA, "Isaac Lab documentation," 2026. Available: https://isaac-sim.github.io/IsaacLab/.

[26] Flutter, "Flutter desktop documentation," 2026. Available: https://docs.flutter.dev/platform-integration/desktop.

[27] FastAPI, "FastAPI documentation," 2026. Available: https://fastapi.tiangolo.com/.

Appendix A GitLab Repository and Running the Project

A.1 Repository

GitLab project URL: https://git.cs.bham.ac.uk/projects-2025-26/sxc1742

The `backend` directory contains the FastAPI orchestration service, robot inspection, configuration services, reinforcement-learning logic, the Isaac subprocess workers and the LLM agent tools. The Flutter desktop application is held under `frontend/rtg-flutter-app`. The `docs` directory holds the architecture notes, setup records and design plans, while `scripts` provides the Windows and Linux launchers, the build tooling and the media and figure generators. `backend/runs` stores the configuration, telemetry, checkpoints and evaluation video for every experiment, and it is the source of every number quoted in Chapter 6.

A.2 Minimum setup

- Install Python 3.12, the Flutter desktop tooling and the backend packages listed in `backend/requirements.txt`.
- For GPU training, install NVIDIA Isaac Sim 6.0.1 and Isaac Lab 3.0 as described in `docs/isaac_setup.md`.
- Create the Python environment using `uv` and install the backend requirements into it.
- Start both services with `scripts/dev_run.ps1` on Windows or `scripts/dev_run.sh` on Linux.
- Load a URDF, configure the goal and reward, run Test reward, then launch Isaac training from the desktop application.

A.3 Verification commands

Backend tests: `$env:PYTEST_DISABLE_PLUGIN_AUTOLOAD=1; .\.venv\Scripts\python.exe -m pytest backend\tests\ -q --basetemp=.pytest_tmp`

Frontend: `cd frontend\rtg-flutter-app; flutter analyze; flutter test`

Report figures: `py -m uv run --python 3.12 --with matplotlib --with numpy python scripts/make_report_figures.py`

Report document: `.venv\Scripts\python.exe scripts\write_report.py`

Appendix B Experiment Coverage

Table 7. Coverage of the 34 analysable Isaac runs by robot and algorithm.

Robot	PPO	SAC	TD3	Total analysable
A1	8	0	0	8
ANYmal	10	0	0	10
Go1	4	3	2	9
Valkyrie	3	3	1	7

Six timestamped run directories contained no numeric reward telemetry and are retained within the failure count rather than discarded. One further directory, named `p3gallop`, does not follow the timestamped `*-isaac` naming convention and is excluded from all fleet-level calculations. Every figure in this report is generated from the run archive at build time by `scripts/make_report_figures.py`, and the run directories themselves remain the authoritative data.

Appendix C Supplementary Figures

These figures support Chapter 6, but they were moved into this appendix to keep the main body within the page limit. Each of them is created from the same run archive as the figures in the body.

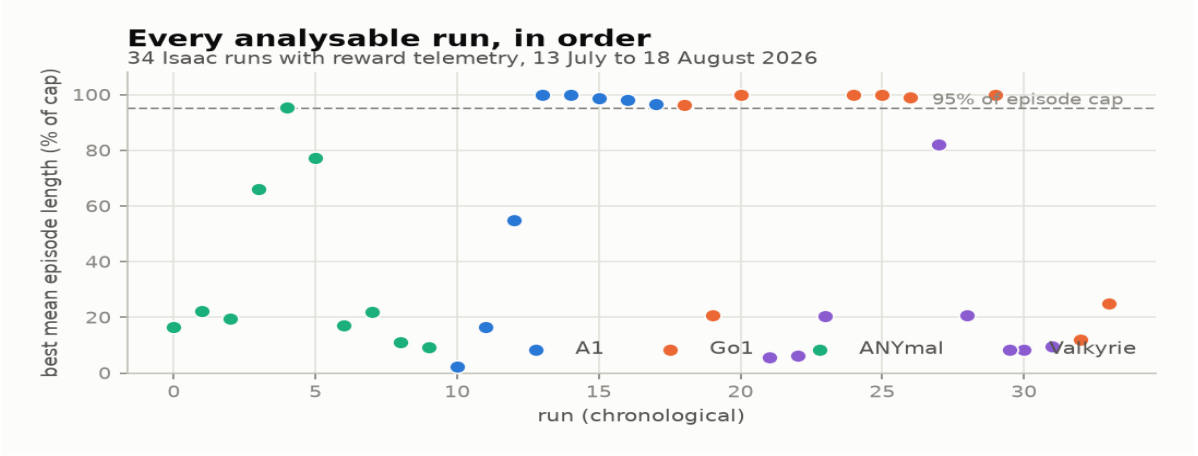


Figure 13. Best mean episode length for every analysable Isaac run, in chronological order.

The chronological view refers to the shape of the project as a whole and not to an outcome. The early ANYmal runs are grouped low, while the A1 and Go1 runs from late July onward are grouped near the cap. The two runs on the far right, both at 16 environments, are the additions made on 18 August, which set the lower end of the throughput relationship discussed in Section 6.4.

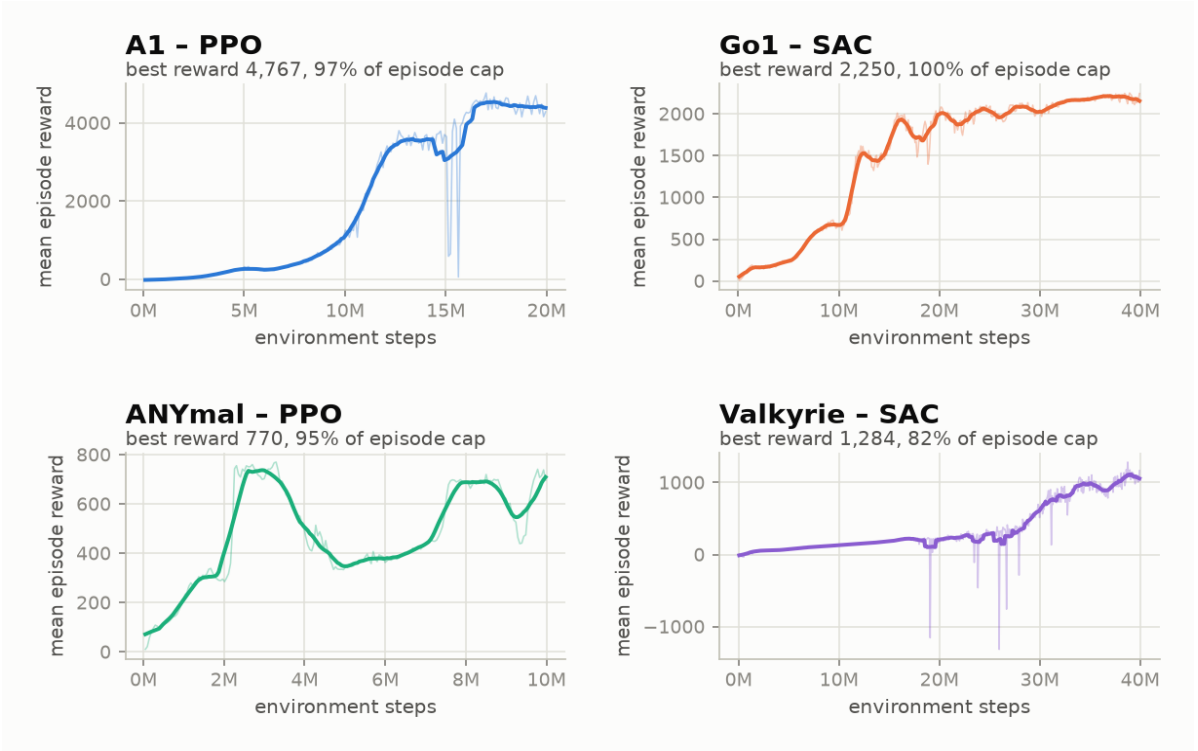


Figure 14. Reward against environment steps for the best run of each robot family.

The four best runs are drawn on the same scale in each panel and illustrate the different behaviours of the four bodies. A1 rises steeply and then jumps again around 16M steps, while Go1 climbs in steps that can be clearly seen. ANYmal is unstable throughout, dipping by half between 3M and 5M steps before recovering, which is the most explicit single example of why

ANYmal needed ten attempts. Valkyrie improves slowly for 25M steps and only then accelerates, which is why a shorter budget would have forced the conclusion that the humanoid could not learn at all.

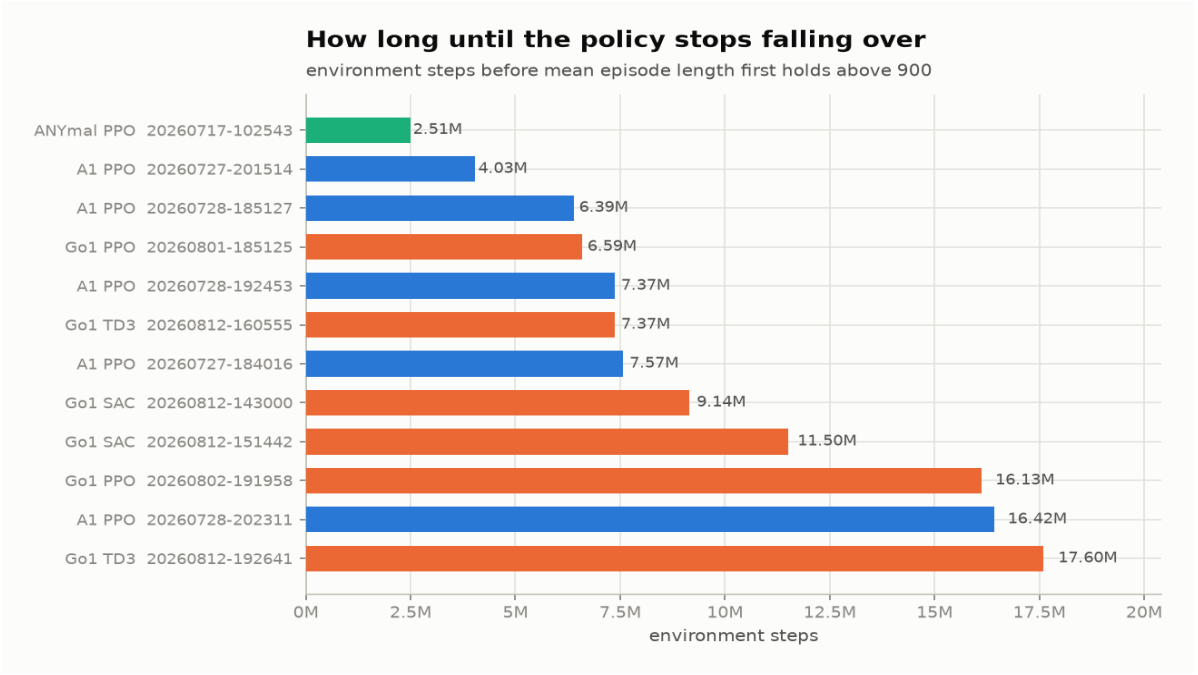


Figure 15. Environment steps before each run's smoothed mean episode length first holds above 900.

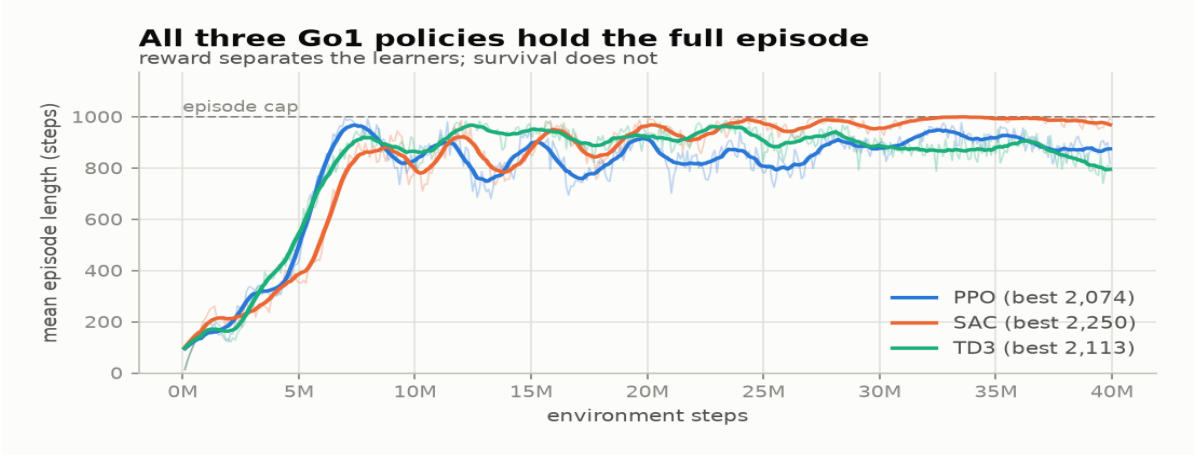


Figure 16. Episode length for the best Go1 run under each learner.

The Go1 episode-length panel is the companion to Section 6.6: all three learners reach and hold the cap, so on this robot survival does not distinguish between them at all. It should be compared against the Valkyrie equivalent in Section 6.8, where survival is the only thing that does.

Appendix D Full Run Index

All the runs in the archive that can be analysed are shown below. Within a run, the reward is the best mean episode reward observed; the episode length is the mean episode length at that same step; and the peak steps per second is the highest throughput recorded during the run. All the numbers quoted at fleet level in Chapter 6 come from this table.

Table 8. All 34 analysable Isaac runs, 13 July to 18 August 2026.

Run	Robot	Algorithm	Envs	Best reward	Episode length	% of cap	Peak steps/s
20260713-083121	ANYmal	PPO	512	137.0	165.9	16.6%	2,095
20260713-084227	ANYmal	PPO	256	193.4	222.2	22.2%	738
20260717-093639	ANYmal	PPO	256	159.1	195.4	19.5%	1,207
20260717-093932	ANYmal	PPO	256	518.0	659.4	65.9%	3,572
20260717-102543	ANYmal	PPO	2048	769.8	954.6	95.5%	21,056
20260717-110526	ANYmal	PPO	128	738.2	771.6	77.2%	591
20260722-133515	ANYmal	PPO	2048	144.0	170.5	17.0%	5,582
20260722-140347	ANYmal	PPO	1024	174.6	219.7	22.0%	2,047
20260722-141517	ANYmal	PPO	1024	93.1	112.0	11.2%	4,745
20260722-141803	ANYmal	PPO	1024	76.7	93.0	9.3%	1,525
20260727-173801	A1	PPO	2048	12.3	22.5	2.2%	12,420
20260727-180733	A1	PPO	2048	152.0	164.2	16.4%	10,966
20260727-182222	A1	PPO	8192	506.5	547.6	54.8%	11,493
20260727-184016	A1	PPO	4096	2,146.8	1000.0	100.0%	17,925
20260727-201514	A1	PPO	4096	2,338.3	1000.0	100.0%	20,285
20260728-185127	A1	PPO	4096	1,865.3	988.3	98.8%	30,599
20260728-192453	A1	PPO	4096	1,900.9	979.9	98.0%	28,462
20260728-202311	A1	PPO	4096	4,767.0	967.5	96.8%	32,057
20260801-185125	Go1	PPO	4096	2,073.9	962.7	96.3%	27,500
20260801-215400	Go1	PPO	2048	147.7	208.8	20.9%	18,081

Run	Robot	Algorithm	Envs	Best reward	Episode length	% of cap	Peak steps/s
20260802-191958	Go1	PPO	6400	1,600.6	1000.0	100.0%	31,631
20260803-211052	Valkyrie	PPO	4096	88.3	55.9	5.6%	14,556
20260803-230732	Valkyrie	PPO	4096	96.1	62.4	6.2%	11,230
20260803-233317	Valkyrie	PPO	5800	276.1	204.2	20.4%	14,280
20260812-143000	Go1	SAC	4096	1,952.9	1000.0	100.0%	26,180
20260812-151442	Go1	SAC	4096	2,250.1	1000.0	100.0%	27,840
20260812-160555	Go1	TD3	4096	2,112.7	990.9	99.1%	28,976
20260812-165024	Valkyrie	SAC	3048	1,283.8	821.0	82.1%	11,172
20260812-181127	Valkyrie	TD3	4096	287.0	206.2	20.6%	14,389
20260812-192641	Go1	TD3	4096	1,905.0	1000.0	100.0%	22,239
20260813-175701	Valkyrie	SAC	6200	251.3	82.8	8.3%	14,182
20260813-190946	Valkyrie	SAC	2048	140.0	94.1	9.4%	8,540
20260818-084821	Go1	SAC	16	97.5	121.1	12.1%	96
20260818-145041	Go1	PPO	16	147.2	250.4	25.0%	80

The twelve runs that hold at least 95% of the episode cap are the five A1 PPO runs from 27 and 28 July, the single ANYmal PPO run of 17 July at 2,048 environments, and all six Go1 runs at 4,096 environments or above. No Valkyrie run reaches that threshold under any learner.

Appendix E Compute Environment and Reproduction Notes

Every training and evaluation result reported here was produced on a single workstation, and the pre-release status of the simulator stack is relevant to any attempt at reproduction.

Table 9. Compute and software environment used for all experiments.

Component	Version or specification
GPU	One NVIDIA RTX 5000
Isaac Sim	6.0.1-rc.7
Isaac Lab	3.0.0-beta2 (`v3.0.0-beta2.patch1`)
Physics backend	Newton (replaces `omni.physics.tensors` from Isaac Sim 6.0)
Isaac Sim Python	3.12, bundled with Isaac Sim
Application Python	3.12, project virtual environment

Component	Version or specification
Inspection simulator	PyBullet, headless
Backend	FastAPI
Desktop client	Flutter
Episode cap	1,000 steps

Isaac Sim is a release candidate and Isaac Lab is a beta, and therefore both components are pre-release. This is important for reproduction because of the switch to the Newton physics backend in Isaac Sim 6.0, so results from an earlier `omni.physics.tensors` build should not be expected to match the ones reported here. Isaac Sim also owns its own bundled Python and therefore cannot run inside the application's virtual environment, which is the practical reason for the subprocess architecture described in Section 4.5, rather than a stylistic preference.

If you are reproducing this work, here are three additional notes which may be helpful. First, the throughput values are peak ones observed during a run, not sustained averages, and they are affected by the environment count, as described in Section 6.4. Secondly, the two runs at 16 environments were deliberately small diagnostic runs, and must not be interpreted as unsuccessful training attempts. Third, every figure in this report is regenerated from `backend/runs` using `scripts/make_report_figures.py`, so when a run is added to the archive and the figure and document scripts are re-run, the evidence is updated without any manual editing.